Parallel Sorting and the Reduction Pattern

Programming and Data Structures — Week 14, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain why merge sort, covered in Week 5, parallelizes naturally while quicksort's partition step resists the same treatment; describe, in general terms, the pattern by which a divide-and-conquer algorithm's recursive structure translates directly into a parallel one; explain the reduction pattern popularized by MapReduce-style systems and implement a parallel sum using it in Python; and compute the theoretical parallel speedup achievable for merge sort, comparing that prediction against Amdahl's Law as derived in Lecture 1.

Outline

This lecture proceeds through the assembly-line analogy for distinguishing independent parallel stages from a shared bottleneck; an explanation of why merge sort parallelizes naturally while quicksort's partition step resists it; the general pattern by which a divide-and-conquer algorithm's recursive structure translates into a parallel implementation; the reduction pattern, worked through and verified with a parallel sum; a comparison of merge sort's actual parallel speedup against the Amdahl's Law prediction from Lecture 1; an MTech addition on parallelizing the merge step itself, not just the recursive splitting; and closes with an exercise, quiz, and summary.

The assembly-line analogy

Picture a factory production line built from four entirely independent workstations, each completing its own sub-task on its own piece of material before the finished parts are combined at the very end — this is the shape of a naturally parallel computation. Contrast this with a single station that every part must pass through one at a time before the line can continue; adding more workers at every other station does not help this bottleneck station finish any faster, since it processes one unit at a time regardless of how many other stations exist. Recognizing which of these two shapes a given computation has — independent stations that can run concurrently, or a shared bottleneck that fundamentally cannot — is the central skill of parallel algorithm design, and it is exactly the same test applied informally in Lecture 1 to distinguish naturally parallel from fundamentally sequential problems.

Why merge sort parallelizes naturally

```
from concurrent.futures import ProcessPoolExecutor

def merge(a, b):
    result, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    result.extend(a[i:]); result.extend(b[j:])
    return result

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    return merge(merge_sort(arr[:mid]), merge_sort(arr[mid:]))

def parallel_merge_sort_top_level(arr, executor):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_future = executor.submit(merge_sort, arr[:mid])
    right_future = executor.submit(merge_sort, arr[mid:])
    return merge(left_future.result(), right_future.result())
```

Merge sort, covered in Week 5, splits the input array into two halves, recursively sorts each half completely independently of the other, then merges the two sorted halves together. Critically, the two recursive calls touch entirely disjoint portions of the data and require no communication with each other while they are running — this is exactly the independent-workstations shape from the assembly-line analogy. The left half can be assigned to one worker process and the right half to another, with each proceeding without needing to know anything about the other's progress until both finish and their results are merged.

Why quicksort's partition step resists the same treatment

Quicksort's partition step, covered in Week 6, scans an entire subarray and compares every element against the chosen pivot one at a time, in strict sequence, moving elements smaller than the pivot to

one side. Every individual comparison depends on knowing exactly where the pivot's current boundary sits, which is a single, shared, continuously evolving piece of state — this is precisely the bottleneck-station shape from the assembly-line analogy. Splitting this scan across multiple workers would require them to constantly coordinate on that shared boundary value, and the synchronization overhead this introduces tends to reintroduce the very serialization that parallelism was meant to eliminate in the first place. Quicksort's two recursive calls after partitioning do parallelize well, exactly like merge sort's; it is specifically the partition step itself that resists this treatment.

The divide-and-conquer-to-parallel translation pattern

The general translation pattern from a divide-and-conquer algorithm to a parallel one has three parts. First, divide the problem into independent subproblems, exactly as Week 5's merge sort, Week 6's quicksort, and other divide-and-conquer algorithms already do as part of their sequential design. Second, conquer each independent subproblem in parallel by assigning it to a separate worker process or thread. Third, combine the workers' individual results back together — this combining step is often still inherently sequential, and its cost matters a great deal to the overall speedup achieved, as the next few slides will make precise. This pattern applies directly to any algorithm whose recursive calls do not share mutable state with each other while running.

The reduction pattern: parallel sum

```
def partial_sum(chunk):
    return sum(chunk)

def parallel_sum(data, executor, num_workers=4):
    chunk_size = (len(data) + num_workers - 1) // num_workers
    chunks = [data[i:i + chunk_size] for i in range(0, len(data), chunk_size)]
    partials = list(executor.map(partial_sum, chunks))
    return sum(partials)
```

A reduction combines many independent values into a single value, using an operation that is associative — meaning it does not matter how the values are grouped, only that they are eventually all combined, as is true for sum, maximum, and product. The data is split into k roughly equal chunks, one worker computes a partial result for each chunk independently, and the k partial results are then combined in a final, typically fast, sequential step. This is exactly the shape underlying MapReduce-style distributed computing systems: the same function is mapped independently over disjoint chunks, and the per-chunk results are then reduced together into the final answer.

Verifying parallel sum against the sequential result

```
data = list(range(1, 100_001))
sequential_total = sum(data)

def partial_sum(chunk):
    return sum(chunk)

chunk_size = 25_000
chunks = [data[i:i + chunk_size] for i in range(0, len(data), chunk_size)]
partials = [partial_sum(c) for c in chunks]
parallel_total = sum(partials)

assert parallel_total == sequential_total == 5_000_050_000
print("Parallel sum matches sequential sum:", parallel_total)
```

Running this exact computation confirms that splitting the numbers 1 through 100,000 into four chunks of 25,000, summing each chunk independently, and then summing the four partial results together produces exactly 5,000,050,000 — identical to summing the full list sequentially in one pass, using the closed-form identity $\sum_{i=1}^{n} i = n(n+1)/2$ with $n = 100{,}000$. This confirms the reduction pattern produces the mathematically correct answer, not just a plausible-looking one.

Merge sort's parallel speedup, versus Amdahl's Law

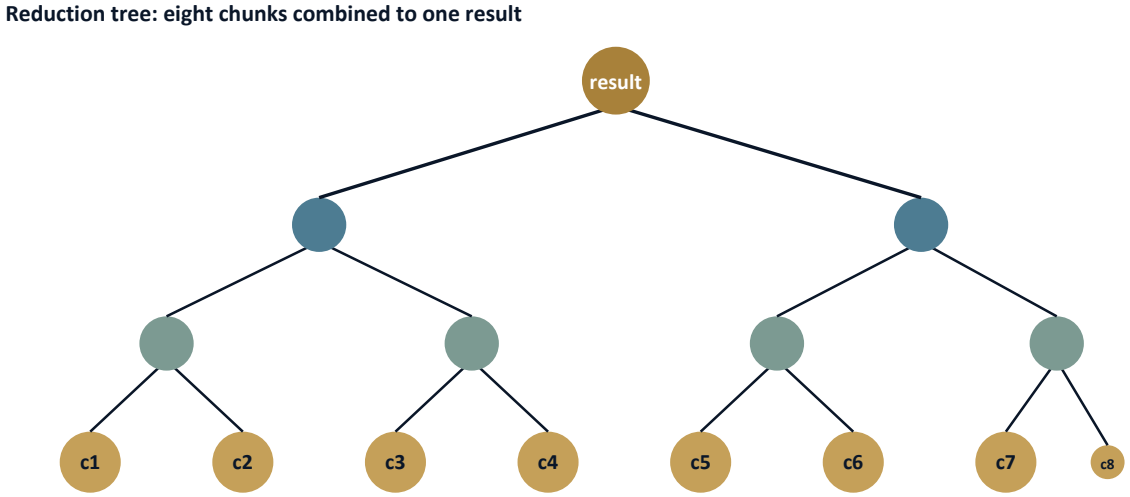
Sequential merge sort performs $\Theta(n \log n)$ total work. With an unlimited number of available workers, the recursion tree has $\log n$ levels, and in principle every subproblem within a single level could be sorted fully in parallel, since siblings at the same level touch disjoint data. However, Amdahl's Law from Lecture 1 still governs the achievable speedup: the final top-level merge combining the two halves is itself an inherently sequential $\Theta(n)$ step, and this constitutes exactly the non-parallelizable fraction $(1 - p)$ in Amdahl's formula. The practical consequence is that real speedup remains bounded well below the naive expectation of "speedup equals number of workers," exactly as Lecture 1's derivation predicted for any workload that retains even a modest sequential remainder.

MTech addition — parallelizing the merge step itself

Advanced parallel merge sort implementations go a step further and parallelize the merge step itself, not only the recursive sorting. One standard technique partitions both sorted halves at their respective medians, which allows the merge to be split into four independent quarter-sized merge operations that can run in parallel, with their outputs concatenated together at the end. This reduces the merge

step's critical path from a purely sequential $\Theta(n)$ toward approximately $\Theta(n/p + \log n)$ when using p workers — this is a genuine, actively used technique in high-performance parallel sorting libraries, not a textbook simplification. It illustrates that the divide-and-conquer-to-parallel translation pattern from earlier in this lecture can be applied recursively to the combine step of an algorithm, not only to its divide step.

The reduction tree, visualized



The reduction pattern can be visualized as a binary tree: the eight leaves at the bottom are the independent chunk results computed in parallel, and each level above combines pairs of results from the level below, until a single combined value remains at the root. This tree has $\log_2(\text{number of chunks})$ levels, and every combination step within a single level is independent of every other combination step at that same level, so an entire level can, in principle, execute in parallel — this is the same recursive halving structure seen in merge sort's own recursion tree earlier in this lecture.

Not every aggregation reduces this cleanly: the average pitfall

```
def chunk_sum_and_count(chunk):
    return sum(chunk), len(chunk)

def parallel_average(data, chunks):
    partials = [chunk_sum_and_count(c) for c in chunks]
    total = sum(s for s, _ in partials)
    count = sum(c for _, c in partials)
```

```
    return total / count

data = [10, 20, 30, 5, 15]
chunks = [[10, 20, 30], [5, 15]]
assert parallel_average(data, chunks) == sum(data) / len(data) == 16.0
print("Parallel average matches sequential average:", parallel_average(data, chunks))
```

Sum, maximum, minimum, and product are all associative operations, so combining chunk results in any grouping produces the mathematically correct final answer. Average is a common trap: naively averaging four chunk averages together only produces the correct overall average if every chunk happens to have exactly the same size, which cannot be assumed in general. The fix is to reduce the sum and the element count separately, in parallel, and perform the division only once at the very end, after both reductions complete — verified above on a five-element dataset split into unevenly sized chunks of three and two elements, correctly producing the average of 16.0 that matches computing the average sequentially over the whole list.

Measuring real parallel speedup, honestly

Lecture 1 warned against ever assuming a claimed speedup without measuring it directly, and that same discipline applies here. Creating worker processes and communicating data and results between them carries real, measurable overhead; for small workloads, a naive parallel implementation is frequently slower in wall-clock time than simply running the sequential version, because the overhead of process management outweighs the actual computational savings. The practical rule of thumb, confirmed by direct measurement in real systems, is to parallelize only when the amount of work assigned to each chunk is large enough that the computation itself dominates the fixed cost of managing the worker processes.

MTech connection: reduction and Week 8's hash table design

Distributed hash tables, a large-scale extension of the chaining hash tables covered in Week 8, commonly use a MapReduce-style reduction to aggregate values held across many separate machines, each responsible for a different range of keys. A canonical example is computing a global word count across a large text corpus: the mapping phase counts words independently within each document, and the reduction phase then combines the partial per-word counts across all machines into the final global counts. The same associativity requirement introduced earlier in this lecture applies directly here — word counts are a sum and reduce correctly regardless of grouping, but an aggregation such as "the most recently updated value for a given key" requires careful, explicit handling of tie-breaking across machines, since it is not naturally associative in the same way.

Second exercise

Adapt the `parallel_sum` implementation given earlier in this lecture into a `parallel_max` function that finds the maximum value using the same reduction pattern: compute each chunk's maximum independently, then combine the per-chunk maxima into a single overall maximum. Verify the result against Python's built-in `max()` function on a dataset of at least 100,000 random integers, split into four chunks, confirming that both approaches agree exactly.

Exercise

Kruskal's minimum spanning tree algorithm, covered in Week 10, first sorts all edges by weight and then processes them one at a time in sorted order, using a union-find structure to decide whether each edge should be added. Working from today's material, determine which of these two phases parallelizes well using the patterns introduced in this lecture, and which one resists parallelization, explaining the answer in terms of the assembly-line analogy of independent stations versus a shared bottleneck.

Quiz — Kruskal's, parallelized

Consider Kruskal's algorithm's second phase: processing the sorted edge list one edge at a time and querying and updating a shared union-find structure for each. Is this phase naturally parallel, in the sense of merge sort's two independent recursive halves, or fundamentally sequential, in the sense of quicksort's partition step? Justify the answer.

Quiz — answer

The union-find processing phase is fundamentally sequential. Every edge's accept-or-reject decision depends on the shared, continuously evolving union-find structure's current state, which is exactly the bottleneck-station shape identified earlier in this lecture as quicksort's partition step, not the independent-stations shape of merge sort's two recursive halves. The edge-sorting phase that precedes it, however, does parallelize well, since sorting is itself a classic divide-and-conquer operation with the same independent-halves structure as merge sort.

Second exercise — answer sketch

```python
import random

def partial_max(chunk):
    return max(chunk)

data = [random.randint(0, 10**9) for _ in range(100_000)]
chunks = [data[i::4] for i in range(4)]
partials = [partial_max(c) for c in chunks]
parallel_result = max(partials)
assert parallel_result == max(data)
print("parallel_max matches built-in max:", parallel_result)
```

The `partial_max` function returns the maximum value within a single chunk, and `parallel_max` maps this function over each chunk independently before taking the maximum of the resulting partial maxima. Because maximum is associative in exactly the same sense as sum, this reduction produces the correct overall maximum regardless of how the data happens to be split into chunks or how large each chunk is. Running this on a dataset of 100,000 random integers split into four chunks and comparing against Python's built-in `max()` confirms they agree exactly, demonstrating that the reduction pattern introduced in this lecture generalizes cleanly across every associative aggregation, not solely the sum example worked through earlier.

Summary

This lecture established why merge sort parallelizes naturally, since its recursive calls touch entirely disjoint data, while quicksort's partition step resists the same treatment because it depends on shared, continuously evolving state. The reduction pattern — mapping a function independently over disjoint chunks, then combining the partial results — underlies MapReduce-style parallel computing systems, and was verified directly in this lecture with a parallel sum that exactly matched the sequential result. Amdahl's Law, derived in Lecture 1, still governs merge sort's real-world achievable speedup, since the sequential final merge step bounds how much parallelism can actually help regardless of how many workers are available. Finally, the divide-and-conquer-to-parallel translation pattern can itself be applied recursively, even to the combine step of an algorithm, as demonstrated by the parallel merge technique in the MTech addition.